

Git y Github

1. Introducción al Control de Versiones

Lectura de los apartados 1.1, 1.2 y 1.3 de este enlace:

The entire Pro Git book, written by Scott Chacon and Ben Straub and published by Apress, is available here: <https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Acerca-del-Control-de-Versiones>

1. Acerca del Control de Versiones
2. Una breve historia de Git
3. Fundamentos de Git

2. Git: instalación

Es posible que `git` ya esté ya instalado en su máquina virtual. Si no:

```
$ sudo apt update
$ sudo apt install git
```

```
h1 {
  color:red;
}
```

3. GitHub

GitHub, Inc. is an Internet hosting service for software development and version control using Git. It provides the distributed version control of Git plus access control, bug tracking, software feature requests, task management, continuous integration, and wikis for every project.

Para la práctica debe tener cuenta creada en GitHub.

3.1. Creación de un repositorio en GitHub

Para la práctica se crea en GitHub un repositorio de nombre `test-md-clases`:

- privado
- con fichero README

4. Acceso GitHub desde local

GitHub ya **no** permite acceder desde línea de comandos con usuario y clave. En su lugar deberá usar el acceso ssh (o un *token* personal, esta segunda forma se explica en el anexo).

4.1. Acceso con ssh

Debe seguir los pasos indicados en [Connecting to GitHub with SSH](#). Es necesario crear claves SSH. Para ello puede seguir estos dos enlaces:

1. [Generación de una nueva clave SSH](#)
2. [Agregar una clave SSH nueva a tu cuenta de GitHub](#)

Vea paso a paso un ejemplo de creación de las claves:

```
$ ssh-keygen -t ed25519 -C "profesor@iesromerovargas.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/alu/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/alu/.ssh/id_ed25519
Your public key has been saved in /home/alu/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:10h/UuuiNSGtpnb7brUoPdyu79mQSyBtsPiQGLgicc
profesor@iesromerovargas.com
The key's randomart image is:
+--[ED25519 256]--+
|  .                  |
| + o      . . .      |
| . E o    o + . . .  |
| . o o o B * . . .  |
|   . + S X = .      |
|       = * +. +      |
|       o +o ++.      |
|       . . . *.o=     |
|       . =+ ** .      |
+----[SHA256]-----+
$ ls -al .ssh/*.pub
-rw-r--r-- 1 alu alu 108 feb 23 09:12 .ssh/id_ed25519.pub
$ cat .ssh/id_ed25519.pub
ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIIbaMp/uhRcLwPz0AbQhooXxQN7IsUMfhqE9KtA0toXa
profesor@iesromerovargas.com
$
```

Luego se [añade la clave SSH a GitHub](#)

Y finalmente puede probar la configuración con el comando `ssh -T git@github.com`

```
$ ssh -T git@github.com
Hi angoies! You've successfully authenticated, but GitHub does not provide
shell access.
$
```

Si todo es correcto ahora puede clonar el repositorio de GitHub en local usando el comando `git clone`. La referencia del repositorio a usar la proporciona Github en el botón "Code"

```
$ git clone git@github.com:angoies/test-md-clase.git
Clonando en 'test-md-clase'...
Enter passphrase for key '/home/alu/.ssh/id_ed25519':
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (12/12), done.
Recibiendo objetos: 100% (15/15), listo.
Resolviendo deltas: 100% (2/2), listo.
remote: Total 15 (delta 2), reused 9 (delta 1), pack-reused 0
$
```

Nota: Puede mantener en cache la autenticación usando `ssh-agent` como se indica en [este enlace](#).

5. Proceso básico desarrollo

Se muestran en este apartado los pasos básicos para trabajar en local y subir los cambios a GitHub.

- `git clone`: descargar el repositorio creado previamente en GitHub

```
$ git clone git@github.com:angoies/test-md-github.git
Clonando en 'test-md-github'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Recibiendo objetos: 100% (3/3), listo.
$ cd test-md-github/
~/test-md-github$ ls
README.md
~/test-md-github$
```

- Trabajo en local: modificar fichero del repositorio, por ejemplo `README.md`

```
~/test-md-github$ code README.md
~/test-md-github$
```

- `git status`: comprobar el estado del repositorio, del área de trabajo y de la zona de seguimiento:

```
~/test-md-github$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios no rastreados para el commit:
(usa "git add <archivo>..." para actualizar lo que será confirmado)
```

```
(usa "git restore <archivo>..." para descartar los cambios en el
directorio de trabajo)
    modificado:      README.md

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
~/test-md-github$
```

- **git add:** añadir a la zona de seguimiento los ficheros modificados que queremos añadir al repositorio

```
~/test-md-github$ git add README.md
~/test-md-github$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
(usa "git restore --staged <archivo>..." para sacar del área de stage)
    modificado:      README.md

~/test-md-github$
```

- **git commit:** añadir los cambios en la zona de seguimiento al repositorio

```
~/test-md-github$ git commit -m "cambios en README para demo"
Author identity unknown

*** Por favor cuéntame quien eres.
Ejecuta

git config --global user.email "you@example.com"
git config --global user.name "Tu Nombre"

para configurar la identidad por defecto de tu cuenta.
Omite --global para configurar tu identidad solo en este repositorio.

fatal: no es posible auto-detectar la dirección de correo (se obtuvo
'alu@xdebian11.(none)')
~/test-md-github$
```

El commit **no se ha realizado**, **git** necesita que configuremos el nombre y correo para añadirlo a la información de confirmación. Esto se puede hacer en modo local (opción por defecto) sólo para el repositorio actual, o en modo global.

En este ejemplo lo hacemos en modo local:

```
~/test-md-github$ git config user.email "jgomez@iesromerovargas.com"
~/test-md-github$ git config user.name "Jose Gomez"
```

Y ahora si se ejecuta el commit:

```
~/test-md-github$ git commit -m "cambios en README para demo"
[main 64f7ed1] cambios en README para demo
1 file changed, 20 insertions(+)
~/test-md-github$
```

- **git push**: sincronizar el estado actual del repositorio local con el de GitHub

```
~/test-md-github$ git push
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Comprimiendo objetos: 100% (2/2), listo.
Escribiendo objetos: 100% (3/3), 431 bytes | 431.00 KiB/s, listo.
Total 3 (delta 0), reusado 0 (delta 0), pack-reusado 0
To github.com:angoies/test-md-github.git
c9a4725..64f7ed1  main -> main
~/test-md-github$
```

- Verificar en la web de GitHub los cambios.

6. Enlaces

- [Git - la guía sencilla](#)
- Vídeos: [Git y GitHub para Poetas](#)

7. Anexos

7.1. Paso de repositorio HTTPS a SSH

Tenga en cuenta que si ya ha clonado el repositorio con HTTPS y desea usar SSH será necesario modificar la configuración de los remotos.

Para ello puede seguir estas indicaciones [Cambiar la URL del repositorio remoto](#)

```
$ git remote -v
> origin https://github.com/OWNER/REPOSITORY.git (fetch)
> origin https://github.com/OWNER/REPOSITORY.git (push)

$ git remote set-url origin git@github.com:OWNER/REPOSITORY.git

$ git remote -v
# Verify new remote URL
> origin git@github.com:OWNER/REPOSITORY.git (fetch)
> origin git@github.com:OWNER/REPOSITORY.git (push)
$
```

7.2. Acceso a GitHub con tokens

Otra forma de acceder a Github es usando su usuario y un PAT (*personal access token*)

En este enlace <https://github.blog/2020-12-15-token-authentication-requirements-for-git-operations/> lo más destacable:

Use of tokens offer a number of security benefits over password-based authentication:

1. **Unique** – tokens are specific to GitHub and can be generated per use or per device
2. **Revocable** – tokens can be individually revoked at any time without needing to update unaffected credentials
3. **Limited** – tokens can be narrowly scoped to allow only the access necessary for the use case
4. **Random** – tokens are not subject to the types of dictionary or brute force attempts that simpler passwords that you need to remember or enter regularly might be

Lo que nos interesa: **crear un token y utilizarlo en lugar de usuario/clave.**

- [Enlace directo a creación de tokens](#)
- [Instrucciones de cómo crear el token](#)

Para esta práctica basta generar un token "clásico" y darle todos los permisos al scope **repo**. Almacene el token en un lugar seguro, lo necesitará a continuación.

Para comprobar que todo es correcto intentar clonar el repositorio usando como *password* el token obtenido:

```
$ git clone https://github.com/angoies/test-md-clase.git
Clonando en 'test-md-clase'...
Username for 'https://github.com': angoies
Password for 'https://angoies@github.com':
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 15 (delta 2), reused 9 (delta 1), pack-reused 0
Recibiendo objetos: 100% (15/15), listo.
Resolviendo deltas: 100% (2/2), listo.
$
```

Cada vez que intente acceder a un repositorio le pedirá usuario y token. Para trabajar con más comodidad es posible almacenar usuario y token en cache usando el comando `git config --global credential.helper 'cache --timeout 3600'` (*timeout* especificado en segundos). Tras ejecutarlo le pedirá autenticarse una vez y esos datos se quedan en cache con ese tiempo de refresco.

```
$ cd test-md-clase/
~/test-md-clase$ git config --global credential.helper 'cache --timeout 3600'
~/test-md-clase$ git push
Username for 'https://github.com': angoies
```

```
Password for 'https://angoies@github.com':  
Everything up-to-date  
~/test-md-clase$ git push  
Everything up-to-date  
~/test-md-clase$
```

Es posible almacenar el token de forma permanente, pero tenga en cuenta que lo hará en texto plano en el fichero `~/.git-credentials`.

```
$ git config --local credential.helper store  
$ git push http://example.com/repo.git  
Username: <type your username>  
Password: <type your password>  
  
[several days later]  
$ git push http://example.com/repo.git  
[your credentials are used automatically]
```

Si quiere personalizar el fichero donde se almacena puede usar la opción `git config --global credential.helper "store --file ~/.my-credentials"`